UNIVERSITY OF PLYMOUTH
School of Engineering, Computing & Mathematics

Lecture 2: Server-side JavaScript with Node.js

Full-Stack Development

Mark Dixon

School of Engineering, Computing and Mathematics

Scrum Standup: Last Session

https://www.atlassian.com/agile/scrum/standups

- What did we do last session

  - Progress:
    did you learn anything (was anything particularly useful or good)?
    (write down topics, what you can remember)

  - Problems / Blockers:

    How did the lab session go (were you able to get something running)?

    What could be better?

Introduction

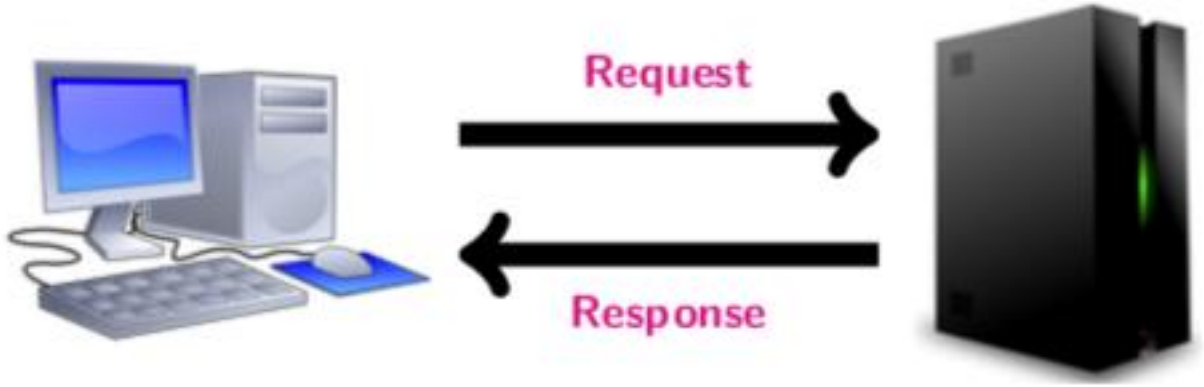
Today's topics

1. Node.js
2. The JavaScript language
3. Functions and events
4. Debugging

Session learning outcomes – **by the end of today's lecture you will be able to:**

- Explain the principles behind the JavaScript language & recognise its common features
- Use browser-based software development tools to debug JavaScript code
- Add backend (server-side) JavaScript to your stack (using Node.js)
- Deploy this using Docker

Client-server architectures



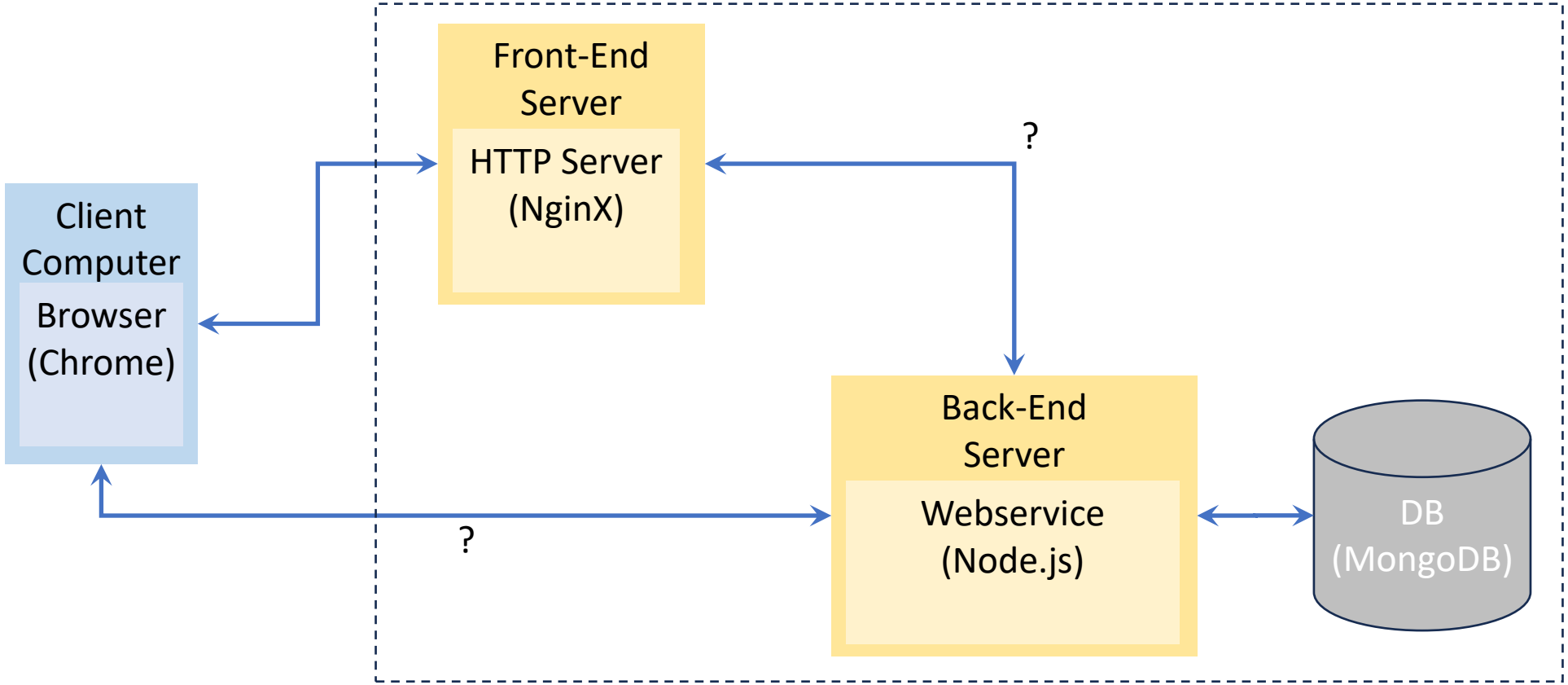
Web servers

A piece of software that receives HTTP requests and serves the appropriate content

- Static web pages
- Dynamic content generated by running a server-side program
- Structured data (e.g. XML, JSON…)
- Images, sound, video…

Full Stack – Context / Component / Container Diagram?

- Sample component architecture (this can vary considerably):



What is the C4 Software Architecture Model?

HTTP protocol

HTTP messages are used to pass information around the web

- Messages have a **header** and **body**
- Header defines
  - Where the message came from
  - Where it is going
  - The payload type (HTML, image, JSON, …)
  - Authentication credentials
  - Many other things…
- Body contains the contents of the message
- HTTP is **stateless** – to preserve state use a cookie (or similar)

HTTP Request Methods: GET or POST

GET

- For fetching static data to display
- Fast
- Returns browser buffered value for identical requests
- Size limited (~2048 characters)
- Data to server passed in URL query string

POST

- For updating server, effecting server-side changes
- Identical calls may produce different responses
- Not buffered
- Slower
- No size limit

PUT, DELETE …

Node.js: Overview

Node.js = **server-side** programming environment **based on JavaScript**

- Node installation sits alongside webserver
  used to produce **dynamic** content
- Event-driven
- Asynchronous non-blocking – good for streaming
- Cross platform
- Many **packages** add functionality

Node.js: Hello World webserver

1. Include required Node modules and define a port
2. Initialise the server, to return a message
3. Listen for requests

require will only work server-side

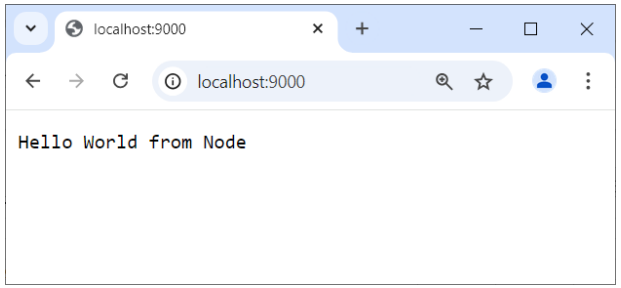
server.js

```
let http = require("http");
let port = 9000;

let server = http.createServer(function(request, response) {
    response.setHeader("Content-Type", "text/plain");
    response.writeHead(200);
    response.end("Hello World from Node");
});

server.listen(port, function() {
    console.log("Server listening on port " + port);
});
```

Node.js: Running the Server

- Option A: run it directly on your host machine (Desktop / laptop)
  - Install Node.js (directly on your host machine)
  - run server application using:

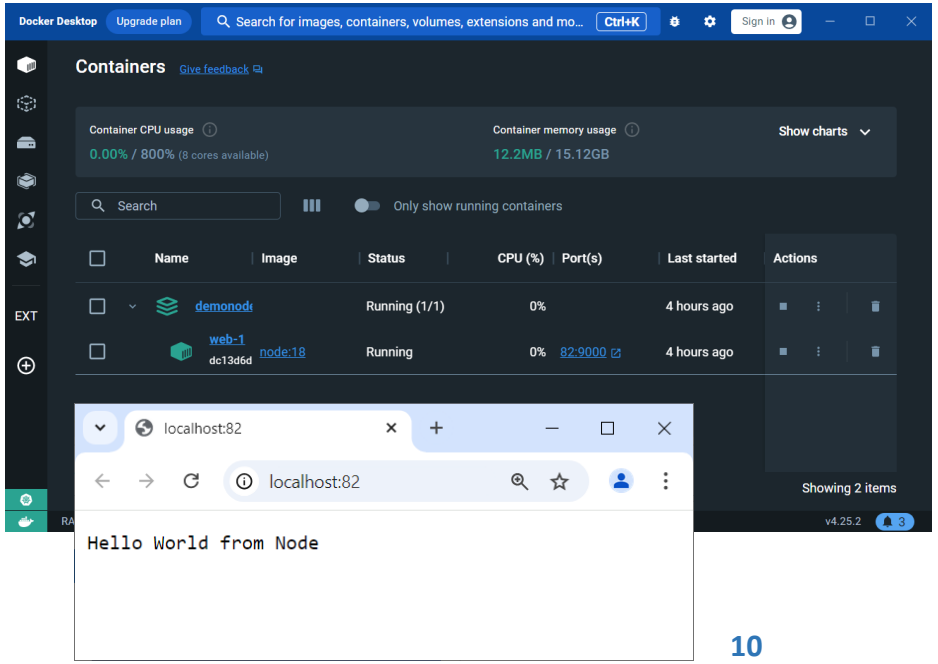
```
node server.js
```



- Option B: use **Docker**

compose.yaml

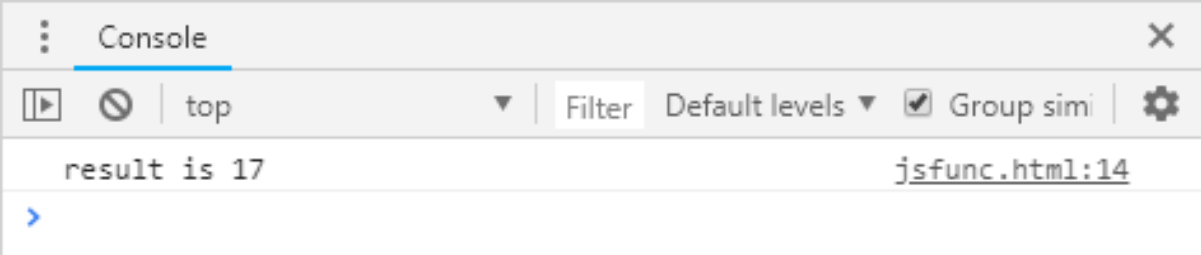
```
services:
  web:
    image: node:22.9.0
    volumes:
      - ./app:/usr/src/app
    ports:
      - "82:9000"
    working_dir: /usr/src/app
    command: sh -c "
      chmod +x run.sh &&
      ./run.sh"
```



Functions

- Have a name (sometimes – can be anonymous)
- Arguments
- Return value

```
function sum(x, y) {
  return x + y;
}

window.onload = function() {
  let result = sum(5, 12);
  console.log("result is " + result);
}
```

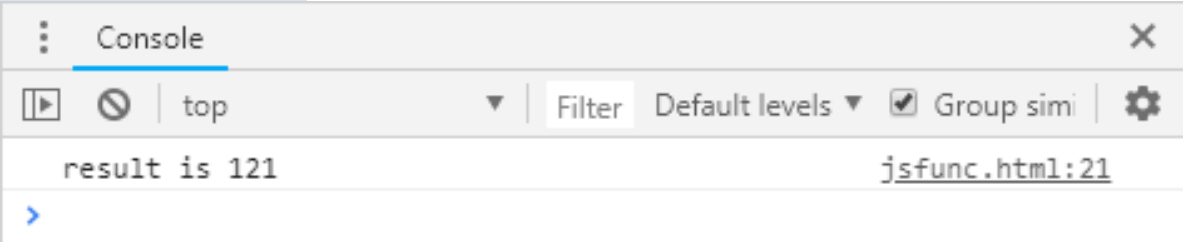


window.onload will only work client-side

Functions

- All functions have an array of arguments – called **arguments**
- Used to pass an arbitrary number of function arguments

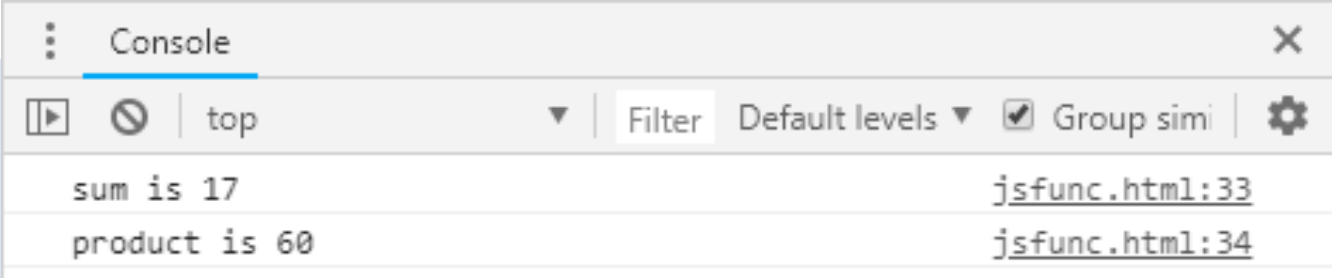
```
function sumArr() {
  let result = 0;
  for (i=0; i<arguments.length; i++) {
    result += arguments[i];
  }
  return result
}

window.onload = function() {
  let result = sumArr(5, 12, 19, 85);
  console.log("result is " + result);
}
```



Function passing

Functions can be passed as arguments to other functions

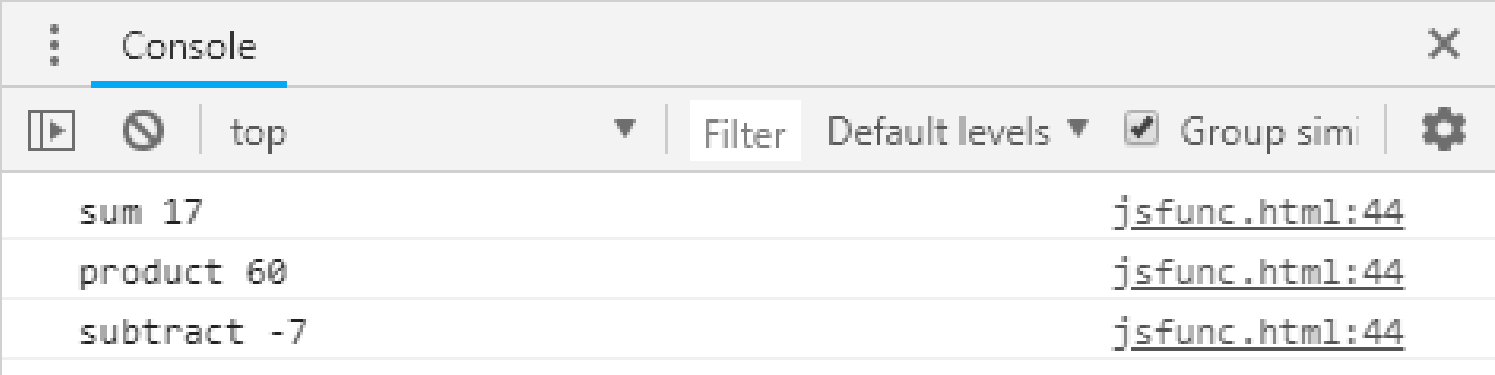
```
function product(x, y) {
  return x * y;
}

function calc(func, x, y) {
  return func(x, y);
}

window.onload = function() {
  console.log("sum is " + calc(sum, 5, 12));
  console.log("product is " + calc(product, 5, 12));
}
```



Arrays of functions

Functions can be stored in arrays

```
function subtract(x, y) {
  return x - y;
}

window.onload = function() {
  let arr = [sum, product, subtract];

  for (i=0; i<arr.length; i++) {
    console.log(arr[i].name + " " + arr[i](5, 12));
  }
}
```



Arrow functions

```
output = (args) => { statements };
```

```
window.onload = function() {
  let square = (x) => { return x * x; }

  console.log(square(4));
  console.log(square(10));
}
```

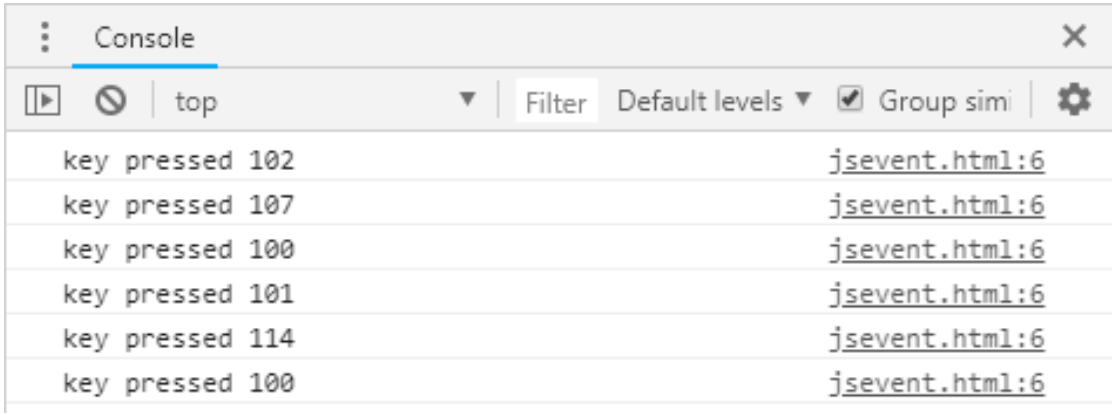
```
function square(x) {
 return x * x;
}
```

- shorter code than normal syntax
- have limitations (such as cannot act as **constructor** for objects)

Events

Asynchronous programming

- JavaScript is an **asynchronous** language
- **Events** fire when something happens (e.g. in the UI) that needs to be handled
- **Functions** handle events
- Attach handlers with **addEventListener** (native JS)




```
window.onload = function {
  let elem = document.getElementById("document");
  elem.addEventListener("keypress", function(evt) {
    console.log("key pressed " + evt.keycode);
  });
}
```

Variable scope – global variables

Variables declared outside of a function are global

- Global variables are visible to **all functions**
- Global variables live for as long as the application is running

Variables that have a value assigned to them before they are declared are **automatically made global**

In HTML global variables belong to the **window object**

```
window.myCounter;
```

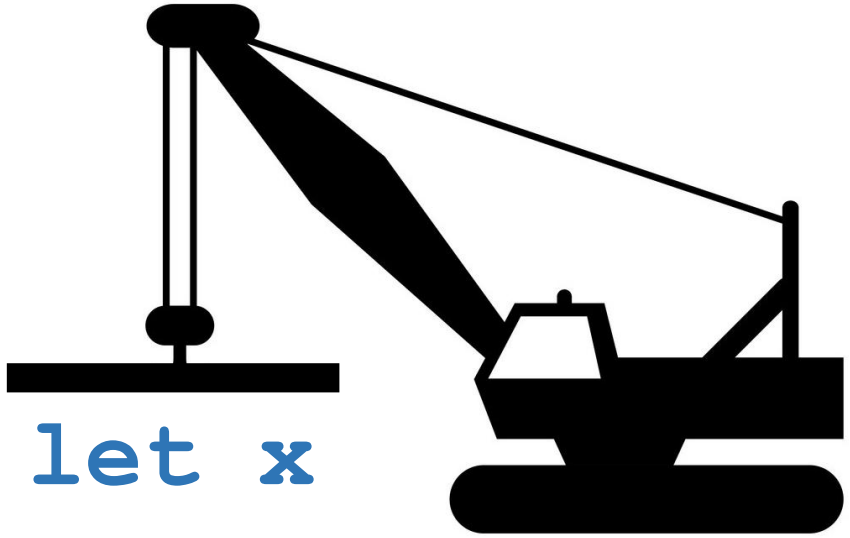
Variable hoisting

Variables declared in the middle of a function exist with a value **undefined** as soon as the function starts executing

- JS moves the variable declaration to the top of the file before execution starts

Referring to a non-existing variable causes an error

Referring to a non-instantiated variable is OK (value is **undefined**)



Variable hoisting

Hoisting example

```
window.onload = function() {
  myVar = "This is my variable"; // variable instantiated on line 2
  alert(myVar);      // referenced on line 3, and declared on line 4
  let myVar;         // The declaration is hoisted & runs first
}
```

Non-instantiated variable example

```
window.onload = function() {
  alert(myVar);                     // variable is called on line 2
  myVar = "This is my variable";   // instantiated on line 3
  let myVar;                      // This causes an error - undefined
}
```

Scope

- Local variables are only visible to some functions
- These functions make up the scope of the variable

No block scope

- JS does not have block scope like C#/Java
- Be careful with loops!
- There is only function scoping
- The **let** keyword introduces block scope

```
window.onload = function() {
  if (true) {
    let x = 1000;
    console.log(x);
  }
  console.log(x);
}
```

JavaScript scoping

- JavaScript gives function objects a **scope object** or **closure** to record local variables
- The closure also contains a link to the closure of **outer** functions to keep their variables in scope
- This link of closures or **scope chain** is defined based on where the functions are **declared**, not where they are **called**

```
window.onload = function() {
    function sum(x) {
        return function(y) {
            return x + y;
        }
    }

    console.log(sum(5)(12));
}
```

Client-side Debugging: runtime code analysis

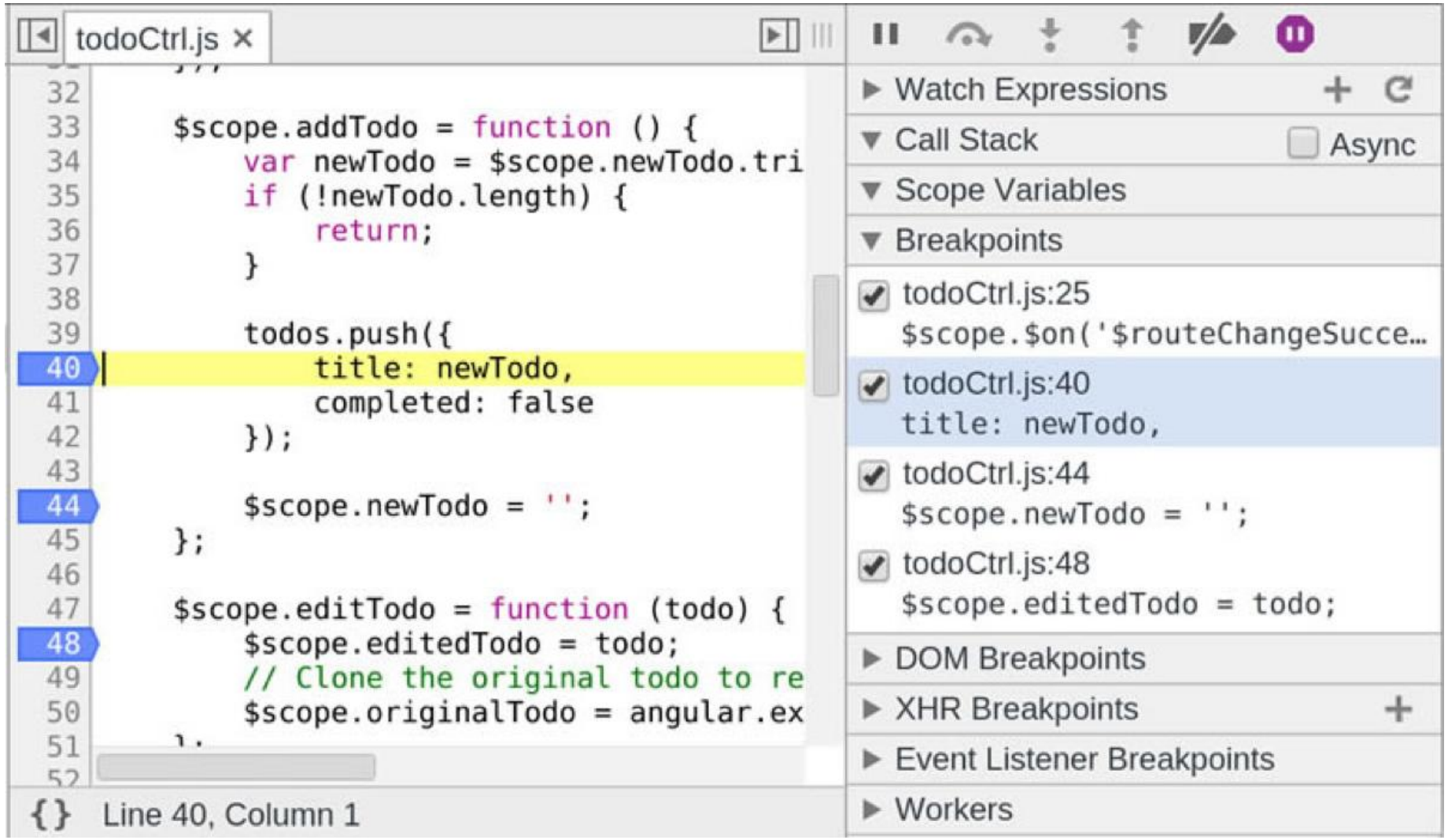
Watch your code while it executes

- Variables
- Exceptions/errors

Debugging with breakpoints

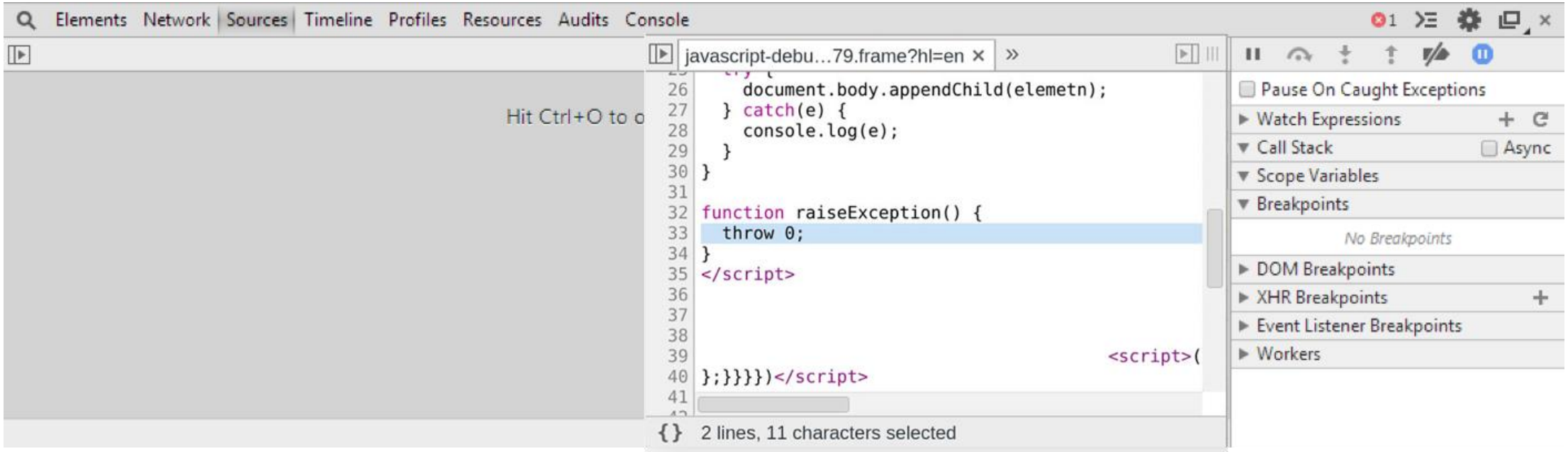
- A **breakpoint** is an intentional stopping or pausing place in a program
- Use breakpoints in DevTools to debug JavaScript, DOM updates and network calls
- See https://developer.chrome.com/devtools/docs/javascript-debugging

Client-side Debugging: Add & remove breakpoints



Client-side Debugging: Pause on exceptions

Click the **Pause on Exceptions** button on the right end of the Source panel to switch to **Pause on all exception** mode

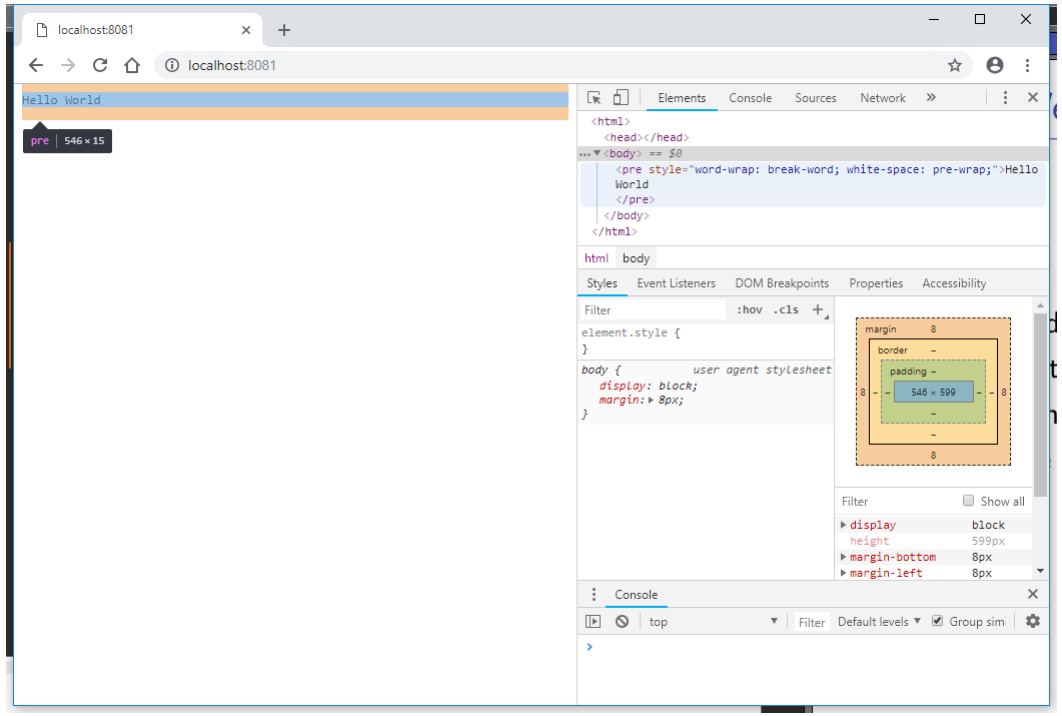


Client-side Debugging: Printing stack traces

Printing log messages to the **DevTools console** is very helpful in understanding how your application behaves

You can make the log entries even more informative by including associated **stack traces** – there are several ways of doing that:

- console.log & Error.stack
- console.trace()



Client-side Debugging: console.log() and Error.stack

events.js

```
983   // Caught a non-IE DOM event. 1 adc
984   var be = new goog.events.BrowserEve
985   try {
986     retval = goog.events.fireListener
987   } catch (e) {
988     console.log(e.stack);
989   } finally {
990     be.dispose();
991   }
992   return retval;
993 };
994
```

Watch Expressions

Call Stack

Not Paused

Scope Variables

Not Paused

Breakpoints

No Breakpoints

DOM Breakpoints

XHR Breakpoints

```
Error: An exception                                                    events.js:988
    at [object Object].triggerForElement (http://www.corp.google.com/~yurys/closure-
library/closure/goog/ui/hovercard.js:209:9)
    at [object Object].handleTriggerMouseOver_
(http://www.corp.google.com/~yurys/closure-library/closure/goog/ui/hovercard.js:186:10)
    at [object Object].handleEvent (http://www.corp.google.com/~yurys/closure-
library/closure/goog/events/listener.js:157:26)
    at Object.fireListener (http://www.corp.google.com/~yurys/closure-
library/closure/goog/events/events.js:753:21)
    at HTMLDocument.<anonymous> (http://www.corp.google.com/~yurys/closure-
library/closure/goog/events/events.js:986:26)
    at HTMLDocument.<anonymous> (http://www.corp.google.com/~yurys/closure-
library/closure/goog/events/pools.js:157:36)
>
```

Client-side Debugging: console.trace()

positioning.js events.js hovercard.js* ×

```
214 goog.ui.HoverCard.prototype.triggerForElement = func
215
216   console.trace();
217   if (anchorElement == this.currentAnchor_) {
218     // Element is already showing, just make sure it
219     this.clearHideTimer();
220     return;
221   }
222
```

{} Line 214, Column 19

Watch Expressions
Call Stack Async
Not Paused
Scope Variables
Not Paused
Breakpoints
dynamicScript.js:2

Console Search Emulation Rendering

<top frame> ▼ <page context> ▼

```
Recompilation and update succeeded.
Recompilation and update succeeded.
Recompilation and update succeeded.
4 ▼console.trace()                                  hovercard.js:216
    goog.ui.HoverCard.triggerForElement             hovercard.js:216
    goog.ui.HoverCard.handleTriggerMouseOver_       hovercard.js:193
    goog.events.fireListener                        events.js:737
    goog.events.handleBrowserEvent_                 events.js:858
    (anonymous function)                            events.js:282
>
```

Catching Errors

Syntax Errors

- The interpreter cannot make sense of the program

Runtime Errors

- *Cannot read properties of null*
- Exceptions

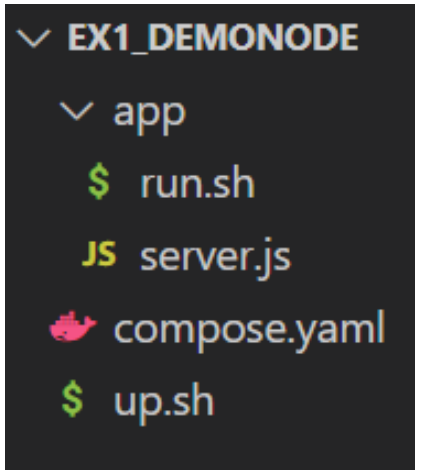
Logical/Semantic Errors

- Your program doesn’t do the right thing

Exercise 1: Single Container Node.js stack using Docker

- The following example uses an Node.js image to deploy your JS server file into a Docker container. You can do everything on the host machine (i.e. your laptop or desktop PC), but this is usually discouraged.
- Create new folder '**demoNode**' (referred to here as the project folder)
- Create new folder '**app**' (inside previous folder)
- Create new file '**server.js**' (inside app folder)
  Use the code provided earlier in these slides.
- Create new file '**run.sh**' (inside app folder)
  Type the following into that file:

```
node server.js
```



Exercise 1: Single Container Node.js stack using Docker

- Create new file '**compose.yaml**' (in project folder)
  Use the code provided earlier is these slides
  (remember the ***indentation*** is critical)

- Create new file '**up.sh**' (inside project folder)
  Type the following into that file:

```
docker-compose up -d
```

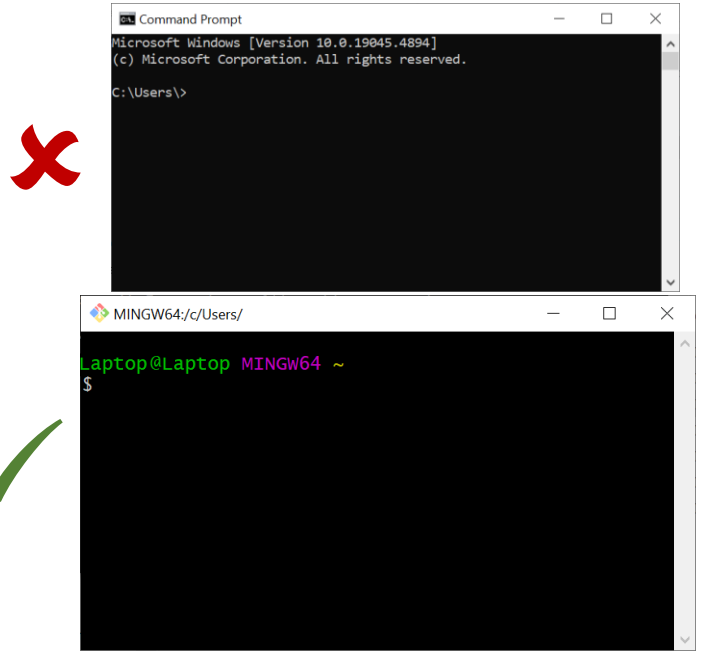
- Terminal into project folder (on your host)
  Start the application up:

```
source up.sh
```

- On Windows and many Linux systems,
  the word source can be shorted to a dot:

```
. up.sh
```

- Navigate to "http://localhost:82/", you should see a Hello message.



Note: these are Unix commands, which will not work in Windows Command Prompt or PowerShell (use **GitBash** or a Unix terminal)

Exercise 2: Greetings

- Alter your server to provide a greeting facility (Good morning / afternoon / evening, depending on the time of day).

Exercise 3: Unique Elements Page

- Create an HTML page that provides the user with a text field. The user should type in numbers separated by commas as shown:
  1,4,3,4,4,7,0,9,3
- Write a function that, when given an array of numbers, finds the number of unique elements in that array. The function should return array of unique elements
- You should then write JavaScript that takes the text input, creates an array from the numbers, and displays the unique elements when a button is clicked

Exercise 4: Two Container Stack

- Attempt to combine your stack from last week with this week's work
  (so that you have 2 containers, an Nginx container and a Node.js container)
  (there should be 2 application folders, but only 1 compose file)

- Attempt to get the containers communicating with each other
  Hint: look for information about Ajax (but don't give this too much time as it will be covered in a subsequent session)
  (think about what functionality can be moved from the Nginx client-side code into your Node.js server-side code)